

Uniswap Continuous Clearing Auction 2.0.0 Audit



Uniswap

June 10, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
2.0.0 Changes	5
Security Model and Privileged Roles	6
Low Severity	8
L-01 requiredDemandQ96AtNextActiveTick Reverts Before First Tick Initialization	8
L-02 Partial forceLateralizeOverTicks Can Artificially Inflate the Clearing Price	8
L-03 Validation Hook Sees Different State For The First Bid In A Block	9
Notes & Additional Information	10
N-01 ValueX7 Could Use User-Defined Operators Instead Of Named Functions	10
N-02 supportsMode Is Not Declared In The Validation Hook Interface	10
N-03 Duplicate Public Getter For The Protocol Fee Controller	11
N-04 Inconsistent Use Of Q96 Suffix For Prices	11
N-05 Unused Import	12
N-06 Missing Docstrings	12
Conclusion	13
Appendix	14
Issue Classification	14

Summary

Type	DeFi	Total Issues	9 (8 resolved)
Timeline	From 2026-05-22 To 2026-05-29	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	3 (3 resolved)
		Notes & Additional Information	6 (5 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the changes in the [Uniswap/continuous-clearing-auction](#) repository between commits [a0aabaa](#) (v1.1.0) and [68e72ac](#).

In scope were the following files:

```
src
├─ ContinuousClearingAuction.sol
├─ ContinuousClearingAuctionFactory.sol
├─ AuctionStorage.sol
├─ CheckpointStorage.sol
├─ StepStorage.sol
├─ lens
│   └─ CCALens.sol
│   └─ TickDataLens.sol
└─ libraries
    └─ CheckpointAccountingLib.sol
    └─ ConstantsLib.sol
    └─ DemandLib.sol
    └─ PriceLib.sol
    └─ ValueX7Lib.sol
```

System Overview

The Uniswap [Continuous Clearing Auction](#) facilitates the launch of new tokens through a continuous clearing mechanism. It establishes a fair clearing price through continuous bidding and scheduled token releases. The central [ContinuousClearingAuction](#) contract may be deployed independently or through the [ContinuousClearingAuctionFactory](#), which initializes parameters such as pricing rules, auction length, issuance phases, and validation procedures. The auction commences at a designated starting block and concludes at a specified ending block, with a set amount of tokens released per block.

Bidders place orders by specifying their maximum acceptable price. Throughout the process, checkpoints capture the auction's state, including the clearing price, cumulative tokens distributed, and total funds collected. The clearing price represents the point where demand aligns with available supply, and it increases as new bids are received.

Upon auction completion, participants can withdraw their bids to retrieve any unused funds or claim acquired tokens, with partial fills utilizing checkpoint data for proportional allocation. Following the auction, the system assesses whether it has achieved graduation by meeting a fundraising threshold. Successful auctions enable the withdrawal of raised currency and the recovery of unsold tokens. Unsuccessful auctions result in full refunds to bidders and the return of all tokens to the recipient. At the designated claim block, buyers may retrieve their allocated tokens to finalize distribution.

2.0.0 Changes

Auction settlement has been reworked to support supply rollover. Demand above the clearing price now carries forward over the remaining issuance schedule, so blocks that release more supply than the demand can absorb credit the unfilled supply against bids that arrive in later blocks. The new [DemandLib](#) and [PriceLib](#) libraries implement the price calculations related to the changes. [DemandLib](#) computes the required demand to clear the supply at a given price, applying a rollover catch-up factor derived from the remaining [mps](#). [PriceLib](#) provides Q96 conversions between currency demand, token amounts, and price.

The factory constructor now requires a [_protocolFeeController](#) address and exposes it via [protocolFeeController\(\)](#); every auction the factory deploys inherits the same

controller. `currencyRaised()` returns the total currency raised including protocol fees, and `lbpInitializationParams()` returns the post-fee `currencyRaised`.

Sweep authorization has been tightened. `sweepCurrency` is now restricted to the configured `FUNDS_RECIPIENT`, and `sweepUnsoldTokens` is now restricted to the configured `TOKENS_RECIPIENT`. Callers other than these recipients revert with the new `NotAuthorized(address authorized, address caller)` error. In `v1.1.0` both sweeps were callable by any address. Additionally, `lbpInitializationParams()` now reverts with `NotGraduated` if the auction did not graduate; previously it returned stale values for unsuccessful auctions.

Two new lens contracts have been added for offchain reads. `CCALens` is a stateless, multicable entry point that composes the `AuctionStateLens` and `TickDataLens` views. `TickDataLens` walks the initialized tick list and returns the demand and the additional currency required at each tick, applying the rollover-aware demand formula. Both are safe to share across all auctions on a chain.

Security Model and Privileged Roles

Throughout the in-scope codebase, the following assumptions and privileged roles were identified:

- The `TOKENS_RECIPIENT` and `FUNDS_RECIPIENT` addresses are fixed at deployment and cannot be changed. Only these accounts can call `sweepUnsoldTokens` and `sweepCurrency` respectively.
- The `IProtocolFeeController` address is fixed at factory deployment and shared by every auction the factory produces. The controller is trusted to return a fee amount no greater than the currency raised. A compromised controller can redirect all raised currency through the fee transfer path. Deploying a new factory is the only way to switch to a different fee controller.
- The currency and the token are assumed to conform to the standard ERC-20 interface and to follow conventional balance semantics. Rebasing tokens, fee-on-transfer tokens, double-entry-point tokens, and tokens with transfer callbacks are not supported.

- Auctions are ownerless and non-upgradeable after deployment. There is no admin, governance, or pause role on either the factory or the auction contracts.
- Auction deployers are assumed to be non-malicious and they are assumed to create auctions with reasonable parameters.
- Tokens deposited into the auction contract beyond the configured total supply are not recoverable.

Additional considerations and integration guidelines can be found in the full [technical reference](#).

Low Severity

L-01 `requiredDemandQ96AtNextActiveTick` Reverts Before First Tick Initialization

The view function `requiredDemandQ96AtNextActiveTick` reads `nextActiveTickPrice` from storage and passes it straight to `DemandLib.requiredDemandAtPrice` without checking whether the value is a real tick price or the `MAX_TICK_PTR` sentinel value.

At deployment, `nextActiveTickPrice` defaults to `MAX_TICK_PTR`. It stays at that value until the first call to `_initializeTickIfNeeded`. While `MAX_TICK_PTR` is in place, the view function overflows and reverts.

Consider returning `0` (or `type(uint256).max`) when `MAX_TICK_PTR` is set, instead of forwarding to the math library.

Update: Resolved in [pull request #345](#) at commit [d32e1fe](#). The team stated:

| Fixed to return `0`.

L-02 Partial `forceIterateOverTicks` Can Artificially Inflate the Clearing Price

The `forceIterateOverTicks` function walks the tick list up to a caller-chosen `_untilTickPrice` and writes the resulting clearing price to storage. The walk itself is delegated to `_iterateOverTicksAndFindClearingPrice`, which returns the price implied by the running `demandAboveClearingQ96` accumulator once the loop exits.

However, when `nextActiveTickPrice` advances to equal `_untilTickPrice` and `clearingPrice` does not already coincide with that price, the loop exits without crossing the tick at `_untilTickPrice`. Since the subtraction that removes a tick's demand from `demandAboveClearingQ96` happens only inside the loop body, crossing is the sole point at which that demand is cleared. When the un-crossed `_untilTickPrice` holds large demand, the price derived from the still-inflated accumulator can therefore exceed `_untilTickPrice`.

itself, and `forceIterateOverTicks` writes this overstated value into the clearing price without inserting a new checkpoint.

This stale, inflated clearing price is then read by `_submitBid`, which rejects any bid whose `_maxPrice` is at or below it. When the current block has already been checkpointed by a prior bid in the same block, `_checkpointAtBlock` short-circuits, so the inflated price is not corrected before the bid-price check runs. In normal blocks, the next block's full checkpoint restores the correct value and the censorship is transient. However, `submitBid` reverts with `AuctionIsOver` once the block number reaches `END_BLOCK`, so no block after the final bidding block accepts bids. As a result, a bidder whose `_maxPrice` sits above the true final clearing price but at or below the inflated value is permanently excluded from the auction.

Consider clamping the value returned by `_iterateOverTicksAndFindClearingPrice` to `_untilTickPrice` whenever the loop exits without crossing that tick, so that a partial iteration cannot push the price above the correct clearing price.

Update: Resolved in [pull request #355](#). The team stated:

We believe that preventing `forceIterateOverTicks` from being called AFTER a checkpoint is made will fix this.

L-03 Validation Hook Sees Different State For The First Bid In A Block

The `_submitBid` flow invokes the [validation hook](#) before it advances the checkpoint through `_checkpointAtBlock`. The hook receives only the bid tuple defined by `IValidationHook.validate`, so a hook that gates on cumulative auction state must read it from storage. Accumulators such as `currencyRaised` are written only inside `_checkpointAtBlock`, and that function is a no-op once the [current block is already checkpointed](#).

As a result, the state observed by the hook depends on the ordering of bids within a block:

1. The first `submitBid` in a new block runs its hook while the accumulators still reflect the previous checkpoint, then advances the checkpoint to the current block.
2. Every subsequent `submitBid` in the same block runs its hook after the checkpoint has already advanced, so it observes the current-block values.

Two otherwise identical bids therefore validate against different cumulative state purely based on their position in the block. A hook that gates on a value like `currencyRaised` would let

the first bid of a block through against the stale (lower) reading while rejecting later bids against the fresh one.

Consider calling `_checkpointAtBlock` before the validation hook in `_submitBid` so that every bid validates against the current-block state, regardless of its ordering within the block.

Update: Resolved in [pull request #346](#).

Notes & Additional Information

N-01 `ValueX7` Could Use User-Defined Operators Instead Of Named Functions

The `ValueX7` type attaches `add`, `sub`, and `min` as named functions through a `using` directive. Solidity has supported [user-defined operators](#) since version 0.8.19; they simplify the code by avoiding custom named functions for common operators. Uniswap v4-core applies this pattern to its `BalanceDelta` and `Currency` types.

Named functions and manual type unwraps are more verbose and obscure otherwise simple operations. For example, `_isGraduated` unwraps both operands to compare them with `>=`, which a comparison operator overload would express directly.

Consider overloading the arithmetic operators (`+`, `-`) for `add` and `sub`, and a comparison operator such as `>=` for the manual unwrap-and-compare patterns, while keeping the existing branchless `min` implementation to preserve its gas savings.

Update: Resolved at commit [ebfd895](#).

N-02 `supportsMode` Is Not Declared In The Validation Hook Interface

The `ValidationHookIntrospection` abstract contract exposes a public `supportsMode` function so that offchain interfaces and integrators can query which validation modes a hook runs. The function is declared only on the concrete contract, and is absent from both the `IValidationHook` interface and the empty `IValidationHookIntrospection` interface.

Integrators that interact through the interface type therefore cannot discover or call it without depending on the concrete implementation.

Consider declaring `supportsMode` in `IValidationHookIntrospection` so the introspection surface is fully described by the interface.

Update: Resolved at commit [f6c5c77](#). The `supportsMode` function has been removed.

N-03 Duplicate Public Getter For The Protocol Fee Controller

The `PROTOCOL_FEE_CONTROLLER` state variable is declared `public immutable`, so the compiler generates a getter for it. The same value is also exposed through the explicit `protocolFeeController` function mandated by `IContinuousClearingAuctionFactory`. The contract thus exposes two public getters that return the same value.

Consider declaring the immutable as `private` or `internal`, since `protocolFeeController` already exposes it through the interface.

Update: Resolved at commit [8093707](#).

N-04 Inconsistent Use Of Q96 Suffix For Prices

All prices in the CCA are Q96 fixed-point values, as stated in the [technical documentation](#). The codebase signals this convention through a `Q96` naming suffix. Some files apply it consistently, such as `DemandLib.sol`, where every price identifier carries the suffix and the units read clearly.

Elsewhere the suffix is applied to some price identifiers and omitted on others holding the same kind of Q96 price within a single scope. In `ContinuousClearingAuction`, the `clearingPriceQ96` local carries the suffix while the `$nextActiveTickPrice` and `$clearingPrice` state variables do not, and `requiredDemandQ96AtNextActiveTick` forwards the unsuffixed `$nextActiveTickPrice` directly into a `_priceQ96` parameter. Because the suffix is present on some identifiers in the same scope, its absence on neighbouring ones reads as meaningful and can suggest the unmarked values use a different representation, which is incorrect.

Consider documenting the Q96 representation once at the contract and interface NatSpec level, and keeping the suffix notation locally consistent within each scope so that neighboring price identifiers do not mix conventions.

Update: Resolved at commit [4e7711e](#).

N-05 Unused Import

The import `import {ConstantsLib} from './ConstantsLib.sol';` in `DemandLib.sol` is unused and could be removed.

Consider removing unused imports to improve the overall clarity and readability of the codebase.

Update: Resolved at commit [05e46b0](#).

N-06 Missing Docstrings

Within `ValidationHookIntrospection.sol`, in the `IValidationHookIntrospection` interface, the docstring is missing.

Consider thoroughly documenting all functions (and their parameters) that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Acknowledged, not resolved.

Conclusion

The `ContinuousClearingAuction` launches new tokens by selling a fixed supply over a block-based issuance schedule and settling all bids at a single uniform clearing price. The `2.0.0` release reworked the settlement math to support supply rollover, so that demand above the clearing price now carries forward over the remaining schedule rather than being settled block by block; the resulting clearing prices and per-block fills may differ from `v1.x` for the same inputs, and the new `DemandLib` and `PriceLib` libraries implement the rollover-aware pricing. The release also threaded an immutable protocol fee controller through the factory and every auction it deploys, so that `sweepCurrency()` now forwards `currencyRaised - protocolFee` to the funds recipient and `lbpInitializationParams()` returns the post-fee amount. In addition, `sweepCurrency()` and `sweepUnsoldTokens()` were restricted to their designated recipients, and the `CCALens` and `TickDataLens` contracts were added for offchain reads.

The findings identified during this audit primarily focused on improving the clarity, consistency, and robustness of the codebase, along with a small number of edge cases in the tick iteration and bid validation flows that could lead to transient or boundary-condition deviations from the intended behavior. No issues affecting the core solvency or fund-custody guarantees of the system were identified. The Uniswap team is commended for their responsiveness and transparency throughout the review process. Their readiness to provide detailed technical explanations and clarify design decisions greatly facilitated the assessment and demonstrated a strong commitment to making their product as secure as possible.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.